

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 5**



SEARCHING

Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin

NIM. 2010817110011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 5

Laporan Praktikum Algoritma & Struktur Data Modul 5: Searching ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin
NIM : 2010817110011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL.....	iv
SOAL.....	1
A. Source Code.....	3
B. Pembahasan.....	4
TAUTAN GIT.....	9

DAFTAR GAMBAR

DAFTAR TABEL

<i>Table 1. Source Code searching_algorithms.cpp.....</i>	<i>4</i>
<i>Table 2. Contoh linear search rekursif.....</i>	<i>5</i>
<i>Table 3. Contoh Binary search rekursif.....</i>	<i>7</i>
<i>Table 4. Table hasil experiment.....</i>	<i>8</i>

SOAL

Sorting

Tujuan Praktikum

1. Mahasiswa mampu mengimplementasikan algoritma searching (Linear Search dan Binary Search).
2. Mahasiswa mampu melakukan analisis performa menggunakan waktu eksekusi (μs) jumlah perbandingan (comparisons), dan indeks hasil pencarian (index_found).
3. Mahasiswa memahami pengaruh distribusi data dan kondisi data terhadap efisiensi algoritma pencarian.

Ketentuan Tugas

- Implementasi dikerjakan secara modular menggunakan template yang disediakan.
- Dilarang keras menggunakan library pencarian bawaan seperti `std::find` atau `std::binary_search`. Semua algoritma harus ditulis manual.
- Dilarang menggunakan AI. Ini kalian harus analisis sendiri. Baca dulu dari sumber-sumber buku, materi, internet yang diberikan.
- Kalau segampang ini masih pakai AI, see you next meet aja.
- Data yang akan dicari berupa `vector<int>`.
- Waktu eksekusi diukur dalam satuan mikrosecond (S).
- Jika elemen tidak ditemukan, `index_found` bernilai -1

Algoritma Searching

- Linear Search
- Binary Search

Instruksi Pengujian

Lakukan pengujian terhadap setiap algoritma menggunakan beberapa kondisi data berikut:

1. Data acak (random) — khusus Linear Search; Binary Search wajib diuji pada data terurut.
2. Data sudah terurut menaik (sorted ascending).
3. Data terurut menurun (sorted descending) — khusus Linear Search.

Untuk setiap kondisi data, lakukan pengujian dengan tiga skenario pencarian:

- Elemen ada di awal (best case position).
- Elemen ada di tengah.
- Elemen tidak ada dalam data (not found).

Hal yang Perlu Dianalisis

Untuk setiap algoritma, lakukan analisis terhadap aspek berikut:

1. Kompleksitas waktu:
 - Best Case
 - Average Case
 - Worst Case
2. Karakteristik algoritma:
 - Apakah algoritma memerlukan data terurut atau tidak.
 - Apakah algoritma bersifat iteratif atau dapat diimplementasikan secara rekursif.
 - Mekanisme utama algoritma dalam melakukan pencarian.
3. Analisis operasi:
 - Jumlah comparison yang dilakukan hingga elemen ditemukan atau seluruh data diperiksa.
 - Nilai `index_found` (indeks elemen yang ditemukan, atau -1 jika tidak ditemukan).
 - Pengaruh posisi elemen target terhadap jumlah comparison.
 - Pengaruh ukuran data (n) terhadap jumlah operasi pada masing-masing algoritma.
4. Analisis performa:
 - Perbandingan waktu eksekusi (s) antar algoritma pada kondisi data yang sama.
 - Algoritma mana yang lebih efisien untuk dataset kecil dan dataset besar.
 - Pengaruh kondisi data (terurut vs acak) terhadap performa masing-masing algoritma.
 - Hubungan antara teori kompleksitas ($O(n)$ vs $O(\log n)$) dan hasil eksperimen yang diperoleh.

A. Source Code

searching_algorithms.cpp

```
1  #include "searching_algorithms.h"
2  #include <chrono>
3
4  using Clock = std::chrono::high_resolution_clock;
5
6  void linearSearch(const std::vector<int> &data, int key,
7  Metrics &m) {
8      for (int i = 0; i < data.size(); i++) {
9          // cek satu persatu
10         m.comparisons++;
11         if (data[i] == key) {
12             m.found_index = i;
13             break;
14         }
15     }
16 }
17
18 void linear_search(const std::vector<int> &data, int
19 target, Metrics &m) {
20     auto start = Clock::now();
21
22     // Done: write the implementation here
23     linearSearch(data, target, m);
24
25     m.time_us =
26         std::chrono::duration<double,
27 std::micro>(Clock::now() - start).count();
28 }
29
30 void binarySearch(const std::vector<int> &data, int key,
31 Metrics &m) {
32     int low = 0;
33     int high = data.size() - 1;
34     while (low <= high) {
35         int mid = low + (high - low) / 2;
36
37         // cek apakah key ada di mid
38         m.comparisons++;
39         if (data[mid] == key) {
40             m.found_index = mid;
```



```

41         break;
42     } else if (data[mid] < key) {
43         low = mid + 1;
44     } else {
45         high = mid - 1;
46     }
47 }
48
49 // Jika tidak ada m.found_index nya default = -1
50 }
51
52 void binary_search(const std::vector<int> &data, int
53 target, Metrics &m) {
54     auto start = Clock::now();
55
56     // Done: write the implementation here
57     binarySearch(data, target, m);
58
59     m.time_us =
60         std::chrono::duration<double,
61 std::micro>(Clock::now() - start).count();
62 }

```

Table 1. Source Code searching_algorithms.cpp

B. Pembahasan

Linear Search

Kompleksitas waktu

1. **Best Case** : $O(1)$. Karena jika target berada di index 0 maka cuma perlu 1 comparison saja.
2. **Average Case** : $O(n)$. Karena rata – rata disini asumsi nya target berada sekitar tengah maka comparasion nya kurang lebih $(n/2)+1$.
3. **Worst Case** : $O(n)$. Karena worst disini artinya target tidak ada di vector data algoritma ini akan terus berjalan sampai $n-1$.

Untuk Big-O notation nya adalah $O(n)$. Karena algoritma ini melakukan for loop sebanyak n . Dengan hasil comparasi 1 di best case dan target+1 comparision di average case dan n comparison di worst.

Karakteristik algoritma

Karena algoritma ini melakukan for loop sebanyak n untuk dan membandingkan nya dengan target, maka algoritma ini tidak memerlukan data yang terurut. Algoritma ini bekerja dengan melakukan iterasi terhadap setiap index dan membandingkan nya dengan target sehingga operasi utama yang digunakan adalah 1 loop sebanyak n dan 1 comparasi didalamnya.

Algoritma yang saya buat ini berbentuk iteratif. Namun algoritma ini bisa saja dibuat dengan rekursif namun ini membuatnya terasa lebih kompleks dari yang seharusnya. Ini contoh code rekursif nya.

Linear search Rekursif

1	void	linear_search_rekursif(const	std::vector<int>
2	&data, int target, int i,		
3		int n, Metrics &m) {	
4	// sudah lewat akhir data / nggak ketemu		
5	if (i >= n) {		
6	return; // found_index = default		
7	}		
8	// Ditaroh ditengah supaya jika sudah habis tidak kena		
9	comparison		
10	m.comparisons++;		
11	// ketemu		
12	if (data[i] == target) {		
13	m.found_index = i;		
14	return;		
15	}		
16	// rekursif		
17	linear_search_rekursif(data, target, i + 1, n, m);		
18	}		

Table 2. Contoh linear search rekursif

Analisis operasi

Comparison : pada best case 1 dan pada worst case sebanyak n (semua data).

Found Index : found index pada best case adalah 1, dan -1 (tidak ketemu) pada worst case.

Pengaruh posisi target pada comparison : pengaruh nya adalah posisi+1, dimana found index 374, comparison 375.

Secara umum besarnya data tidak berpengaruh dalam menentukan banyaknya operasi karena pengaruh posisi target lebih dominan, pada best dan average case jika posisi target 20 dan data 1000, maka comparison nya tetap 21. Bahkan tidak berpengaruh dengan waktu nya,

dimana pada index 0 dengan $n=1000$ $\text{time}=0.130$, $n=10000$ $\text{time}=0.040$. Namun Besar data mempengaruhi worst case dimana target berada terakhir dengan $n=1000000$ $\text{target}=999999$ $\text{comparison}=1000000$ $\text{waktu}=5405.547$. Jadi ya besar data hanya berpengaruh pada worst case.

Binary Search

Kompleksitas waktu

1. **Best Case** : $O(1)$. Jika posisi target ada di tengah vector maka cuma 1 comparasi dan kompleksitasnya jadi konstan.
2. **Average Case** : $O(\log n)$. Karena posisi target acak dan tidak ditengah vector maka range pencarian di vector akan terus dibagi 2 sampai target ketemu di tengah range itu. Dimana hasil nya diharapkan kurang dari $\log n$.
3. **Worst Case** : $O(\log n)$. Karena data tidak ada di vector pada algoritma ini hanya bisa terjadi jika target melebihi n . maka worst case nya sudah pasti $\log n$ pas

Karena algoritma ini merupakan divide and conquer dimana vector data akan dibagi 2 maka secara umum Big-O notation nya jika tidak ada variable lain yang mempengaruhinya adalah $O(\log n)$.

Karakteristik algoritma

Karena algoritma ini membanding kan nilai index mid pada range vector terhadap target untuk mentukan besaran range nya dia memerlukan data yang berurut, sebagai contoh jika target lebih kecil dari mid maka high (belakang) diturun kan ke posisi mid, dan mid akan dikalkulasi ulang. Dari sini saja sudah terlihat bahwa algoritma ini perlu data yang urut. Karena itu operasi utama dari algoritma ini adalah 1 loop while (jika rekursi tidak ada ini) dan 1 comparasi. Dimana loop while sebagai iterasi utama dan comparasi untuk cek target terhadap relatif posisi nya dari mid dan juga mengubah range pencarian.

Algoritma yang saya buat di kode ini bersifat iteratif, tetapi ya algoritma ini ada versi rekursif nya juga. Namun perbedaan nya disini adalah rekursif memiliki kompleksitas ruang $O(\log n)$ karena dia akan rekursif sebanyak $\log n$ (data vector dibagi 2 terus) dan ini menimbulkan keperluan memori di callstack. Dan ini contoh fungsi nya :

Binary search Rekursif

1	void	binary_search_rekursif(const	std::vector<int>
2	&data, int target, int low,		
3		int high, Metrics &m) {	

4	// ruang pencarian habis / tidak ketemu
5	if (low > high) {
6	return; // found_index default
7	}
8	int mid = low + (high - low) / 2;
9	m.comparisons++;
10	if (data[mid] == target) {
11	m.found_index = mid;
12	return;
13	} else if (data[mid] < target) {
14	// Cari ke kanan
15	binary_search_rekursif(data, target, mid + 1, high,
16	m);
17	} else {
18	// Cari ke kiri
19	binary_search_rekursif(data, target, low, mid - 1,
20	m);
21	}
22	}

Table 3. Contoh Binary search rekursif

Analisis operasi

Comparison : pada best case 1 dan pada worst case sebanyak log n (semua data).

Found Index : found index pada best case adalah 1, dan -1 (tidak ketemu) pada worst case.

Pengaruh posisi target pada comparison : posisi data cukup berpengaruh terhadap banyaknya operasi karena jika target berada di tengah vector atau pada $n/2!$ (n/anggota dari faktorial 2) yang factor nya tidak terlalu tinggi maka level rekursi / iterasi nya tidak terlalu banyak.

Besarnya data tidak berpengaruh dalam menentukan banyaknya operasi karena pengaruh posisi target lebih dominan, namun sama dengan linear banyak data disini mempengaruhi worst case tetapi untuk binary karena notasinya log n data besar akan teratasi. Contoh n = 1000000 comparasinya cuma = 20.

Analisis performa

	Linear Search	Binary Search
Average N=1000 Comp	375	3
Average N=1000 Time	2.044 us	0.080 us

Worst N=1000 Comp	1000	10
Worst N=1000 Time	5.350 us	0.130 us
Average N=1000000 Comp	374541	15
Average N=1000000 Time	2042.802 us	0.290 us
Worst N=1000000 Comp	1000000	20
Worst N=1000000 Time	5405.547 us	0.251 us

Table 4. Table hasil experiment

Waktu Linear Search jauh lebih lambat dari Binary Search untuk besar seperti $N=1.000.000$, Linear bisa ratusan dan ribuan mikrodetik sedangkan Binary cuma sekitar dibawah satu mikrodetik. Jauh banget beda waktu untuk kedua algoritma ini untuk dataset besar. Tapi untuk N kecil (misal 1000), beda nya tidak terlalu jauh

Untuk dataset kecil linear masih lumayan, namun yang menjadi keunggulan nya adalah dia tidak perlu data yang terurut. Untuk dataset besar binary search jauh lebih baik. Perbedaan antara $O(N)$ dan $O(\log N)$ akan sangat terasa saat n nya jutaan.

Pengaruh case pada linear search terasa di worst case (target tidak ada) membuat comparisons naik drastis jadi n . Sedangkan pada binary search worst case comparisons cuma naik jadi $\log N$, dimana disini sangat kecil.

Berdasarkan table hasil experiment diatas bisa kita simpulkan bahwa untuk linear search pertumbuhan waktu nya mendekati $O(n)$ dan comparison nya adalah di $O(n)$. Sedangkan waktu binary search hampir konstan meskipun N bertambah besar. Saat N dilipatkan 1000 kali dari 1.000 ke 1.000.000, waktu hanya naik sekitar 1.9x sedangkan comparison nya naik 2x. Ini sangat sesuai dengan teori $O(\log n)$ di mana pertumbuhan waktu sangat lambat seiring bertambahnya N .

TAUTAN GIT

Repo lengkap : <https://github.com/biahlil/Algoritma-dan-Struktur-Data.git>
Linked List : <https://github.com/DSA26-ULM/module-5-searching-biahlil>